🛠️ Manual Técnico – Mérida Activa

1. Introducción

Este documento detalla la arquitectura de software, el stack tecnológico, el modelo de datos y los requisitos del sistema de la aplicación web **Mérida Activa**. Está dirigido a desarrolladores, administradores de sistemas y al tribunal evaluador del proyecto.

2. Arquitectura y Stack Tecnológico

El proyecto sigue una arquitectura **Cliente-Servidor (BaaS - Backend as a Service)**, separando claramente la capa de presentación de la lógica de negocio y persistencia de datos.

2.1. Capa de Presentación (Frontend)

- **Librería Principal:** React 18.
- **Lenguaje:** TypeScript (garantiza el tipado estático, reduciendo errores en tiempo de ejecución).
- **Entorno de Construcción:** Vite (ofrece tiempos de compilación ultrarrápidos y HMR).
- **Enrutamiento:** React Router DOM v6 (gestión de navegación tipo SPA - *Single Page Application*).
- **Estilos y UI:** Tailwind CSS (framework de utilidades CSS para diseño 100% responsive y personalizado).
- **Iconografía:** Bootstrap Icons.

2.2. Capa de Datos y Backend (Supabase)

- **Base de Datos:** PostgreSQL (relacional, robusta y escalable).
- **Autenticación:** Supabase Auth (gestión de JWT, sesiones y encriptación de contraseñas).

- **Seguridad:** RLS (*Row Level Security*) nativo de PostgreSQL para proteger las tablas a nivel de fila.

3. Requisitos del Sistema

3.1. Requisitos Funcionales (RF)

Definen qué debe hacer el sistema.

- **RF-01 (Autenticación):** El sistema debe permitir a los usuarios registrarse, iniciar sesión y cerrar sesión.
- **RF-02 (Catálogo):** El sistema debe mostrar una lista de eventos culturales obtenida de la base de datos.
- **RF-03 (Favoritos):** Un usuario autenticado debe poder añadir y eliminar eventos de su lista personal de favoritos.
- **RF-04 (Reseñas):** Un usuario autenticado debe poder publicar comentarios y otorgar una puntuación (1-5) a los eventos.
- **RF-05 (Gestión de Perfil):** El usuario debe poder modificar su nombre público de visualización y consultar su historial de interacciones.

3.2. Requisitos No Funcionales (RNF)

Definen cómo debe funcionar el sistema (calidad, rendimiento, seguridad).

- **RNF-01 (Rendimiento):** El tiempo de carga inicial de la aplicación no debe superar los 3 segundos gracias a la compilación optimizada de Vite.
- **RNF-02 (Usabilidad):** La interfaz debe ser intuitiva y adaptarse dinámicamente a dispositivos móviles, tablets y escritorio (Mobile First).
- **RNF-03 (Seguridad):** Las contraseñas no deben almacenarse en texto plano. Se delega la encriptación a Supabase Auth.
- **RNF-04 (Integridad de Datos):** Un usuario no puede guardar el mismo evento en favoritos dos veces (Restricción UNIQUE en base de datos).

4. Modelo de Datos

La base de datos relacional está compuesta por las siguientes tablas principales:

Tabla: users (Gestionada por Supabase Auth)

Contiene la información de identidad.

- id (UUID, Primary Key)
- email (Varchar, Unique)
- encrypted_password (Varchar)

Tabla: eventos

Almacena el catálogo de actividades. | Columna | Tipo de Dato | Restricciones | | :--- | :--- | :--- | | id | UUID | Primary Key, Default gen_random_uuid() | | titulo | Varchar | Not Null | | descripcion | Text | Not Null | | imagen_url | Text | Not Null | | fecha | Timestamp | Not Null | | categoria | Varchar | Not Null | | precio | Numeric | Default 0 |

Tabla: favoritos (Tabla Pivote)

Relaciona a los usuarios con los eventos que han guardado. | Columna | Tipo de Dato | Restricciones | | :--- | :--- | :--- | | id | UUID | Primary Key | | usuario_id | UUID | Foreign Key -> users.id | | elemento_id | UUID | Foreign Key -> eventos.id | | tipo_elemento| Varchar | Not Null, Default 'evento' |

Nota Técnica: Se aplica una restricción UNIQUE(usuario_id, elemento_id, tipo_elemento) para evitar duplicidades en el guardado.

Tabla: comentarios

Columna	Tipo de Dato	Restricciones
`id`	UUID	Primary Key
`usuario_id`	UUID	Foreign Key -> `users.id`
`evento_id`	UUID	Foreign Key -> `eventos.id`
`texto`	Text	Not Null
`puntuacion`	Integer	Check (puntuacion >= 1 AND puntuacion <= 5)
`created_at`	Timestamp	Default `now()`

Exportar a Hojas de cálculo

5. Seguridad y Control de Acceso (RLS)

Uno de los pilares técnicos del proyecto es la implementación de **Row Level Security (RLS)** en PostgreSQL a través de Supabase. Esto asegura que la lógica de seguridad resida en la base de datos y no dependa exclusivamente del Frontend.

Ejemplo de Política de Seguridad en la tabla favoritos:

SQL

```
CREATE POLICY "Usuarios gestionan sus propios favoritos"
ON favoritos FOR ALL
TO authenticated
USING (auth.uid() = usuario_id)
WITH CHECK (auth.uid() = usuario_id);
```

Explicación: Esta regla garantiza que un usuario malintencionado no pueda borrar ni ver los favoritos de otro usuario interceptando peticiones de red, ya que el ID del token JWT (auth.uid()) debe coincidir obligatoriamente con el usuario_id de la fila.

6. Estructura y Flujo del Frontend

El proyecto de React sigue una estructura modular para maximizar la reutilización de código:

Plaintext

```
src/
├── componentes/
|   ├── BotonFavorito.tsx   # Componente aislado con lógica de guardado
|   └── FormularioReseña.tsx # Gestión de estados del formulario (1-5 estrellas)
├── context/
|   └── AuthContext.tsx     # Context API para mantener el estado de sesión global
├── pages/
|   ├── Eventos.tsx         # Hace fetch() de todo el catálogo
|   ├── DetalleEvento.tsx   # Consulta por ID y carga comentarios anidados
|   └── MiPerfil.tsx        # Consulta relacional de usuario -> favoritos -> eventos
```

└── supabaseClient.ts # Inicializa la conexión mediante variables de entorno

Gestión de Estados asíncronos

Se ha utilizado el patrón useEffect combinado con funciones asíncronas (async/await) para el Data Fetching. Para optimizar el rendimiento y evitar bucles infinitos (memory leaks), se aplican *Custom Hooks* como useCallback en componentes complejos como MiPerfil.tsx, asegurando que las peticiones a Supabase solo se lancen cuando las dependencias cambian.

7. Despliegue y Variables de Entorno

El paso a producción requiere la configuración de variables de entorno para mantener ocultas las claves de la API.

Archivo .env.local:

Proceso de Build: Al ejecutar npm run build, Vite compila el código TypeScript a JavaScript puro, minifica los assets (CSS/JS) y genera una carpeta /dist estática y altamente optimizada, lista para ser desplegada en plataformas en la nube como Vercel o Netlify.